Limited Time Offer: Get upto 40% off on AlgoMaster Premium. Check it out →

Design LRU Cache

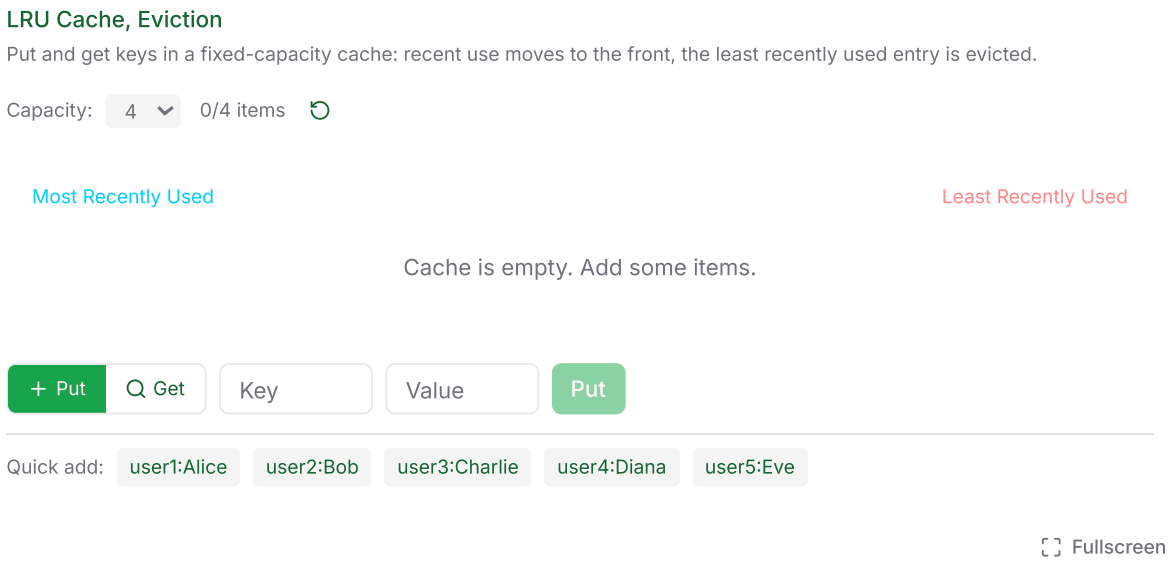
High Priority | 22 min read | Updated June 27, 2026

Quick Revision | Practice this Problem

Listen to this chapter | Unlock Audio

What is a LRU Cache?

LRU stands for **Least Recently Used**. LRU Cache is a type of cache replacement policy that **evicts the least recently accessed item** when the cache reaches its capacity.



In performance-critical systems (like web servers, databases, or OS memory management), caching helps avoid expensive computations or repeated data fetching. But cache memory is limited so when it's full, we need a policy to decide **which item to remove**.

LRU chooses the **least recently accessed** item, based on the assumption that:

> "If you haven't used something for a while, you probably won't need it soon."

The LRU strategy is both intuitive and effective. It reflects real-world usage patterns. We tend to access the same small subset of items frequently, and rarely go back to older, untouched entries.

This makes LRU a popular default caching policy in many systems where **speed and memory efficiency** are critical.

Example

Suppose we have an LRU cache with `capacity = 3`, and we perform the following operations:

```
put(1, A)     // cache: {1:A}
put(2, B)     // cache: {1:A, 2:B}
put(3, C)     // cache: {1:A, 2:B, 3:C}
get(1)        // access key 1 → makes it most recently used
put(4, D)     // key 2 is least recently used → evict it
```

Final cache state: `{3:C, 1:A, 4:D}` (in order of usage from least to most recent)

In this chapter, we will explore the **low level design of a LRU cache.**

Lets start by clarifying the requirements:

---

1. Clarifying Requirements

Before starting the design, it's important to ask thoughtful questions to uncover hidden assumptions, clarify ambiguities, and define the system's scope more precisely.

Here is an example of how a discussion between the candidate and the interviewer might unfold:

DISCUSSION

Candidate: "Should the LRU Cache support generic key-value pairs, or should we restrict it to specific data types?"

Interviewer: "The cache should be generic and support any type of key-value pair, as long as keys are hashable."

Candidate: "Should the cache operations be limited to `get` and `put`, or do we also need to support deletion?"

Interviewer: "For now, we can limit operations to `get` and `put`."

Candidate: "What should the `get` operation return if the key is not found in the cache?"

Interviewer: "You can return either `null` or a sentinel value like `-1`."

Candidate: "How should we handle a `put` operation on an existing key? Should it be treated as a fresh access and move the key to the most recently used position?"

Interviewer: "Yes, an update through `put` should count as usage. The key should be moved to the front as the most recently used."

Candidate: "Will the cache be used in a multi-threaded environment? Do we need to ensure thread safety?"

Interviewer: "Good question. Assume this cache will be used in a multi-threaded server environment. It must be thread-safe."

Candidate: "What are the performance expectations for the `get` and `put` operations?"

Interviewer: "Both `get` and `put` operations must run in `O(1)` time on average."

After gathering the details, we can summarize the key system requirements.

Functional Requirements

- Support `get(key)` operation: returns the value if the key exists, otherwise returns `null` or `-1`
- Support `put(key, value)` operation: inserts a new key-value pair or updates the value of an existing key
- If the cache exceeds its capacity, it should automatically **evict the least recently used item**.
- Both `get` and `put` operations should update the **recency** of the accessed or inserted item.
- Keys and values should be **generic** (e.g., `<K, V>`), provided the keys are hashable.

Non-Functional Requirements

1. **Time Complexity:** Both `get` and `put` operations must run in O(1) time on average.
2. **Thread Safety:** The implementation must be thread-safe for use in concurrent environments.
3. **Modularity:** The design should follow object-oriented principles with clean separation of responsibilities.
4. **Memory Efficiency:** The internal data structures should be optimized for speed and space within the defined constraints.

Now that we understand what we're building, let's identify the building blocks of our system.

---

2. Identifying Core Entities

Unlike systems that model real-world concepts (such as users, products, or bookings), the design of an LRU Cache is centered around choosing the right data structures and in

stractions to achieve the required functionality and performance.

The core challenge here is twofold:

1. We need **fast key-based lookup** for cache reads and updates
2. We need **fast ordering** to track item usage and enforce eviction based on recency

Let's walk through our requirements and identify what needs to exist in our system.

The Need for Fast Lookup

> "Both `get` and `put` operations must run in O(1) time"

To efficiently retrieve values by key, we need a data structure that supports constant-time key access.

The natural choice is a **HashMap**. It provides O(1) average-case lookup and update. When someone calls `get("user_123")`, we need to find that entry immediately, not scan through a list.

But here's the problem: a HashMap doesn't maintain any order. It can't tell us which entry was accessed least recently. If we only used a HashMap, we'd have to scan all entries to find the LRU item during eviction, making that operation O(n).

The Need for Fast Ordering

> "If the cache exceeds its capacity, automatically evict the least recently used item"

The second challenge is maintaining the recency order of cache entries so we can:

- Move a recently accessed item to the front (marking it as Most Recently Used, or MRU)
- Remove the least recently used item from the back when the cache exceeds capacity
- Insert new items at the front (they're considered most recently used)
- Perform all of these operations in O(1) time

An array won't work because inserting or moving elements from the middle involves shifting, which is O(n). A regular linked list won't work either because finding a specific node to move requires O(n) traversal.

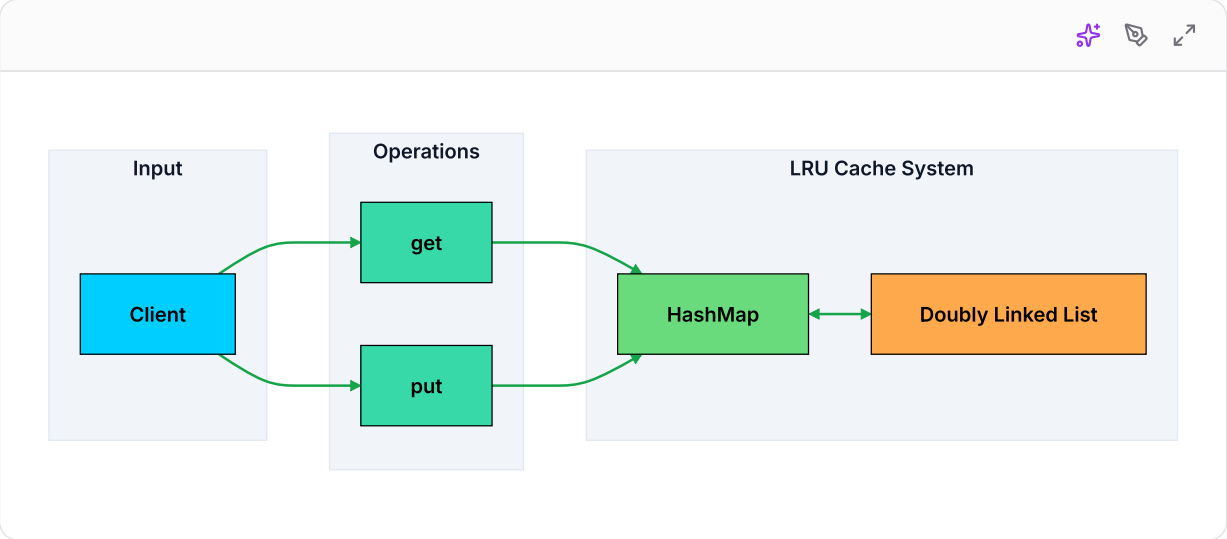
The key insight is using a **Doubly Linked List**. Each node maintains references to both its `prev` and `next` nodes, allowing us to:



- Remove a node from the list in O(1) if we have a direct reference to it
- Move a node to the head in O(1)
- Evict the least recently used node from the tail in O(1)

But how do we get a direct reference to a node without traversing the list? That's where the HashMap comes back into play.

Combining Both Structures

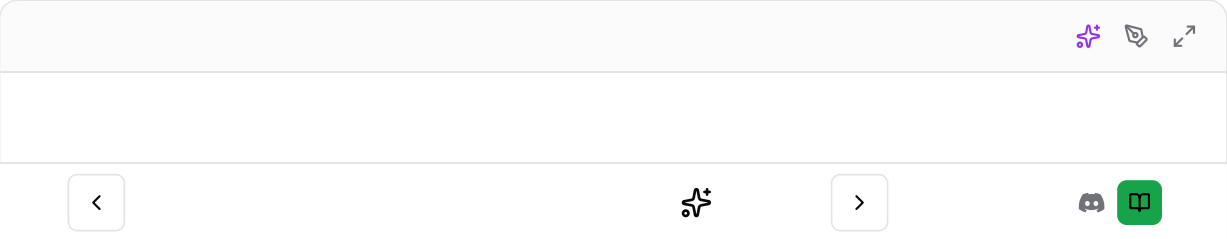


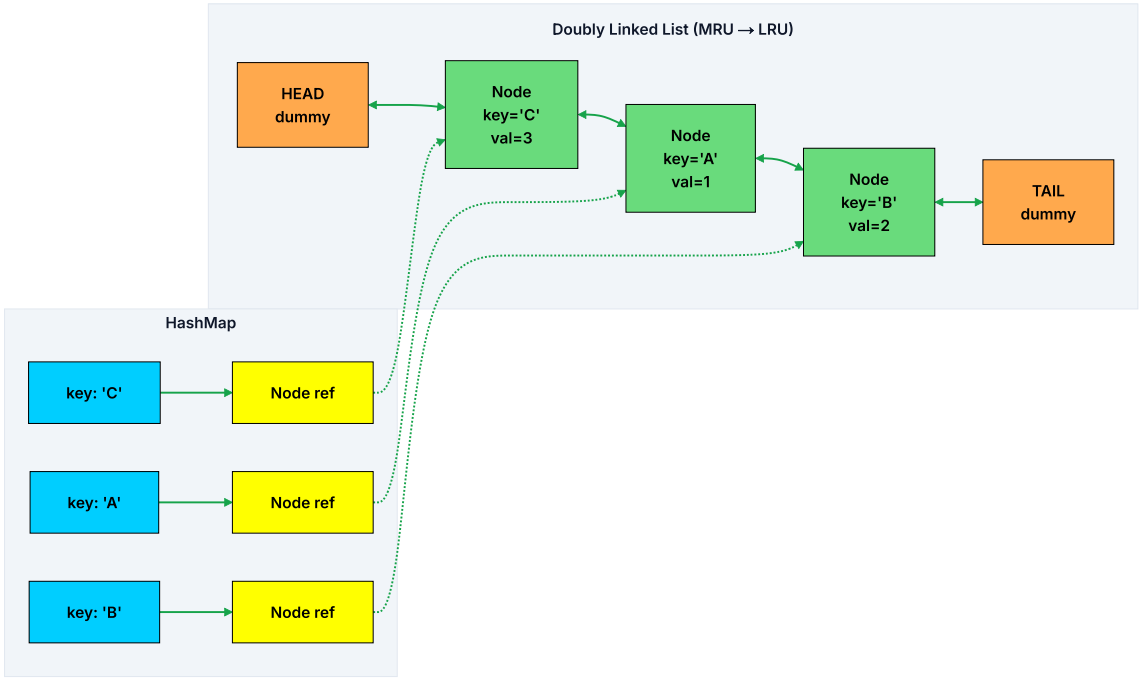
The magic of achieving O(1) for both `get` and `put` lies in combining a HashMap and a Doubly Linked List:

- **HashMap:** Provides O(1) lookup. Instead of storing the value directly, it stores a **pointer/reference to the node** in our Doubly Linked List.
- **Doubly Linked List:** Maintains the usage order. The head of the list is always the Most Recently Used (MRU) item, and the tail is the Least Recently Used (LRU) item.

This combination gives us the best of both worlds:

- To **find** an item, we use the HashMap to get a direct pointer to its node in O(1)
- To **reorder** the item (make it the MRU), we use that pointer to move the node to the head in O(1)
- To **evict** an item, we remove the node from the tail in O(1)





Additional Core Entities

Beyond the HashMap and Doubly Linked List, we need two more classes to encapsulate and organize our logic:

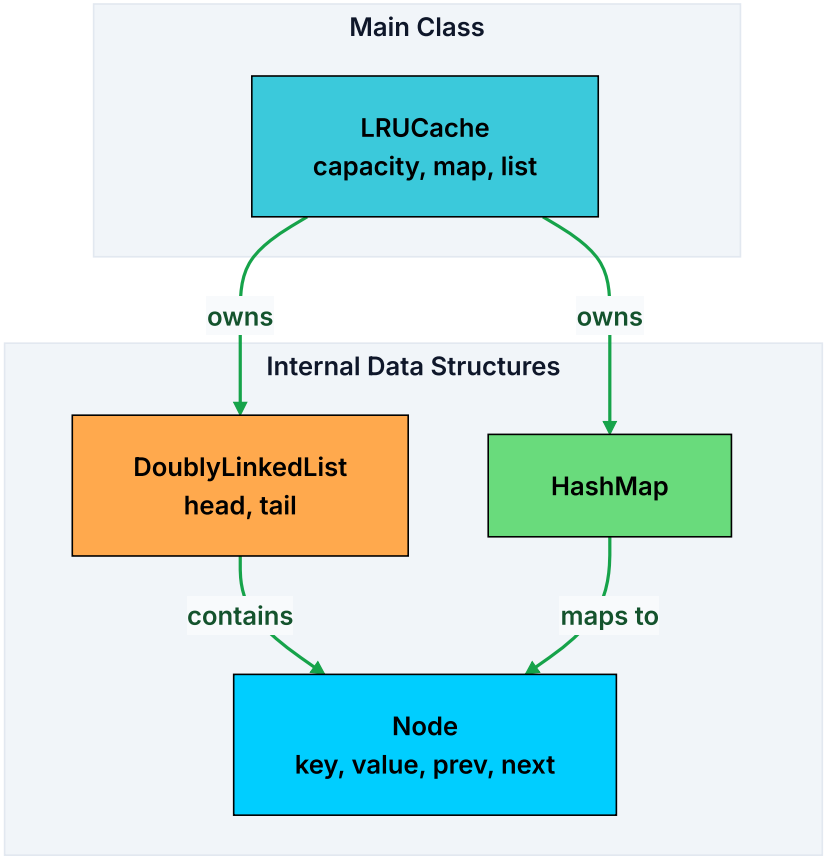
- `Node`: A simple internal class that represents an individual entry in the cache and a node in the linked list. It stores the key-value pair and maintains pointers to adjacent nodes.
- `LRUCache`: The main class that exposes the public cache API and coordinates all operations. It owns both the HashMap and the DoublyLinkedList.

Why store the key in the Node?

When we evict from the tail, we need to remove the entry from the HashMap too. The HashMap needs the key to remove an entry, so each Node must remember its key.

Entity Overview

Here's how these entities relate to each other:



Entity	Type	Responsibility
`Node<K, V>`	Data Class	Stores key-value pair and maintains linked list pointers
`DoublyLinkedList<K, V>`	Utility Class	Manages MRU to LRU ordering with O(1) operations
`HashMap<K, Node>`	Standard Library	Provides O(1) key-to-node lookup
`LRUCache<K, V>`	Main Class	Coordinates all operations and enforces eviction policy

These entities form the core abstractions of our LRU Cache. They enable the system to maintain a fixed-size cache, support constant-time access and updates, and evict the least recently used entry when necessary.

With our entities identified, let's define their attributes, behaviors, and relationships.

---

3. Designing Classes and Relationships

Now that we know what entities we need, let's flesh out their details. For each class, we'll define what data it holds (attributes) and what it can do (methods). Then we'll look at how these classes connect to each other.

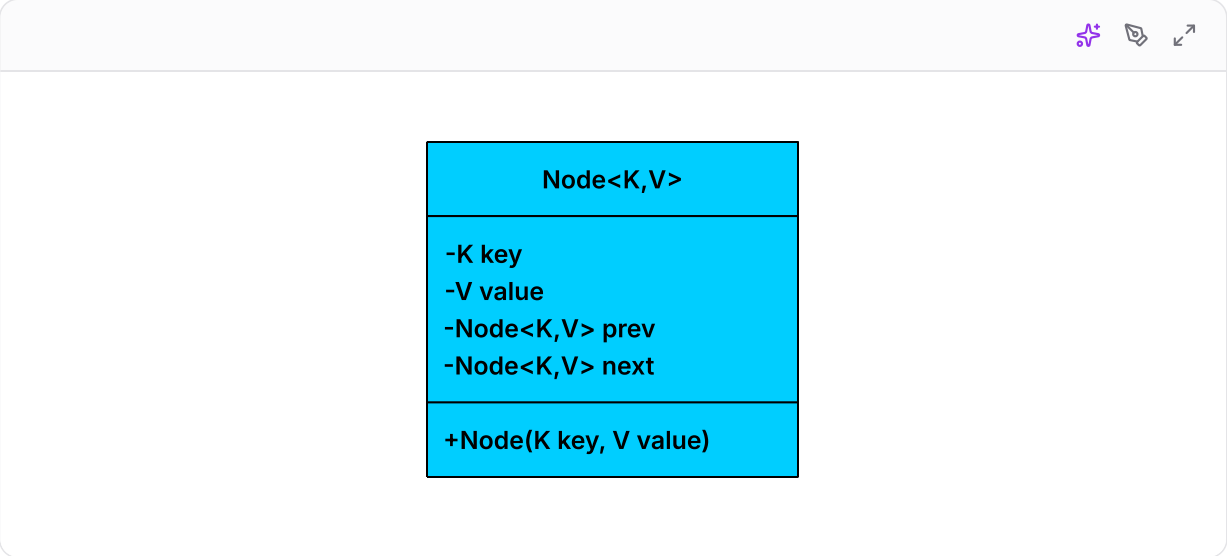
> NOTE
>
> While listing class methods, we will skip trivial **getters and setters** to keep the walk-through focused on core behaviors

3.1 Class Definitions

We'll work bottom-up: simple types first, then the classes with real logic. This order makes sense because complex classes depend on simpler ones.

`Node<K, V>`

The Node class represents an individual entry in the cache and serves as a node in the doubly linked list.



Attribute	Type	Description	Mutable?
`key`	K	The cache key (needed for HashMap removal during eviction)	No
`value`	V	The cached value	Yes
`prev`	Node<K, V>	Reference to previous node in the list	Yes
`next`	Node<K, V>	Reference to next node in the list	Yes

Method	Description
`Node(key, value)`	Constructor that initializes key and value

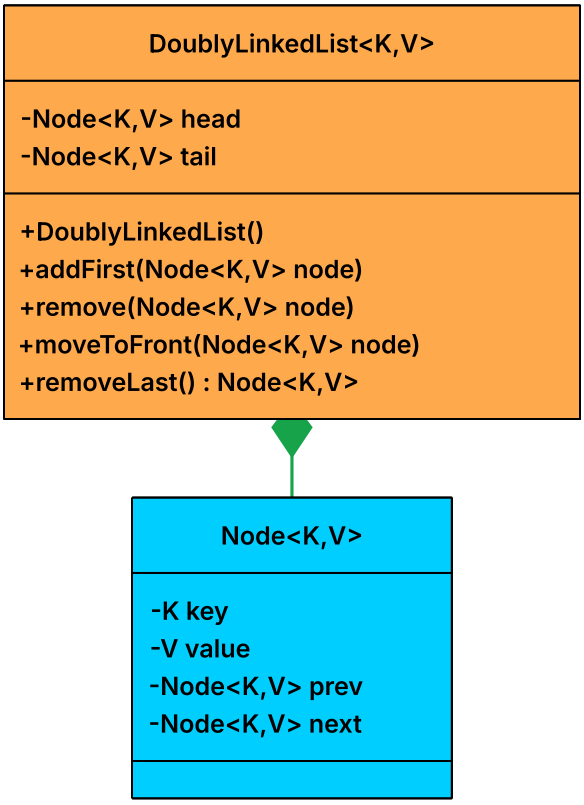
> Design Decision
>
> Why store the key in the Node?

When we evict the LRU item from the tail, we need to remove it from the HashMap. The HashMap's `remove()` method requires the key. Without storing the key in the Node, we'd need to iterate through the HashMap to find which key maps to this Node, making eviction O(n).

The Node class is intentionally simple. It's a data container with minimal behavior. The prev and next pointers are mutable because the node's position in the list changes as items are accessed.

`DoublyLinkedList<K, V>`

A utility class that manages the MRU (Most Recently Used) to LRU (Least Recently Used) ordering of cache entries.



Attribute	Type	Description
`head`	Node<K, V>	Dummy head node for easy insertion (never contains real data)
`tail`	Node<K, V>	Dummy tail node for easy deletion (never contains real data)

Method	Description
`DoublyLinkedList()`	Constructor that creates dummy head and tail, links them together
`addFirst(node)`	Adds a node right after the head (most recently used position)
`remove(node)`	Detaches a node from its current position in the list
`moveToFront(node)`	Removes and re-adds a node to the front
`removeLast()`	Removes and returns the least recently used node (just before the tail)

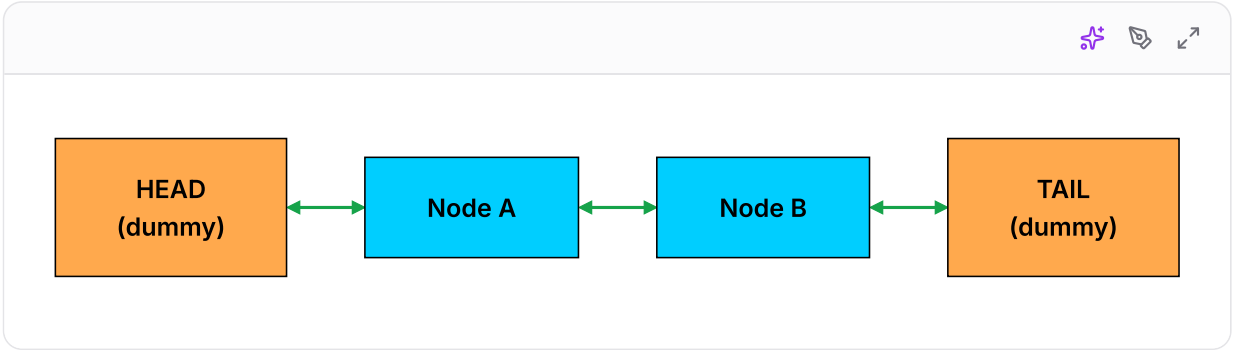
Design Decision

Why use dummy head and tail nodes?

Without them, we'd need special cases for:

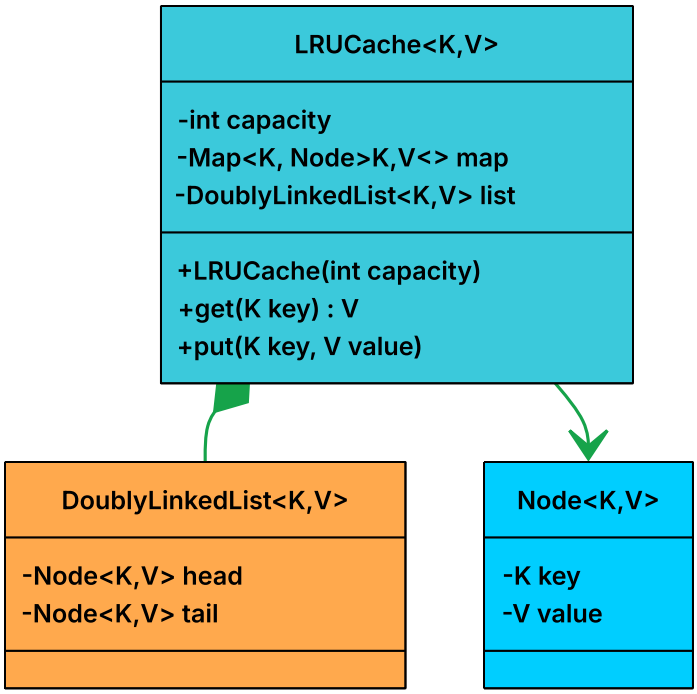
- Adding to an empty list
- Removing the only node
- Removing the first or last node

The dummy nodes eliminate all edge cases. The real data always lives between head and tail, so every real node always has a valid `prev` and `next` pointer.



`LRUCache<K, V>`

The main class that provides the public API (`get` and `put`) and manages the overall cache logic.



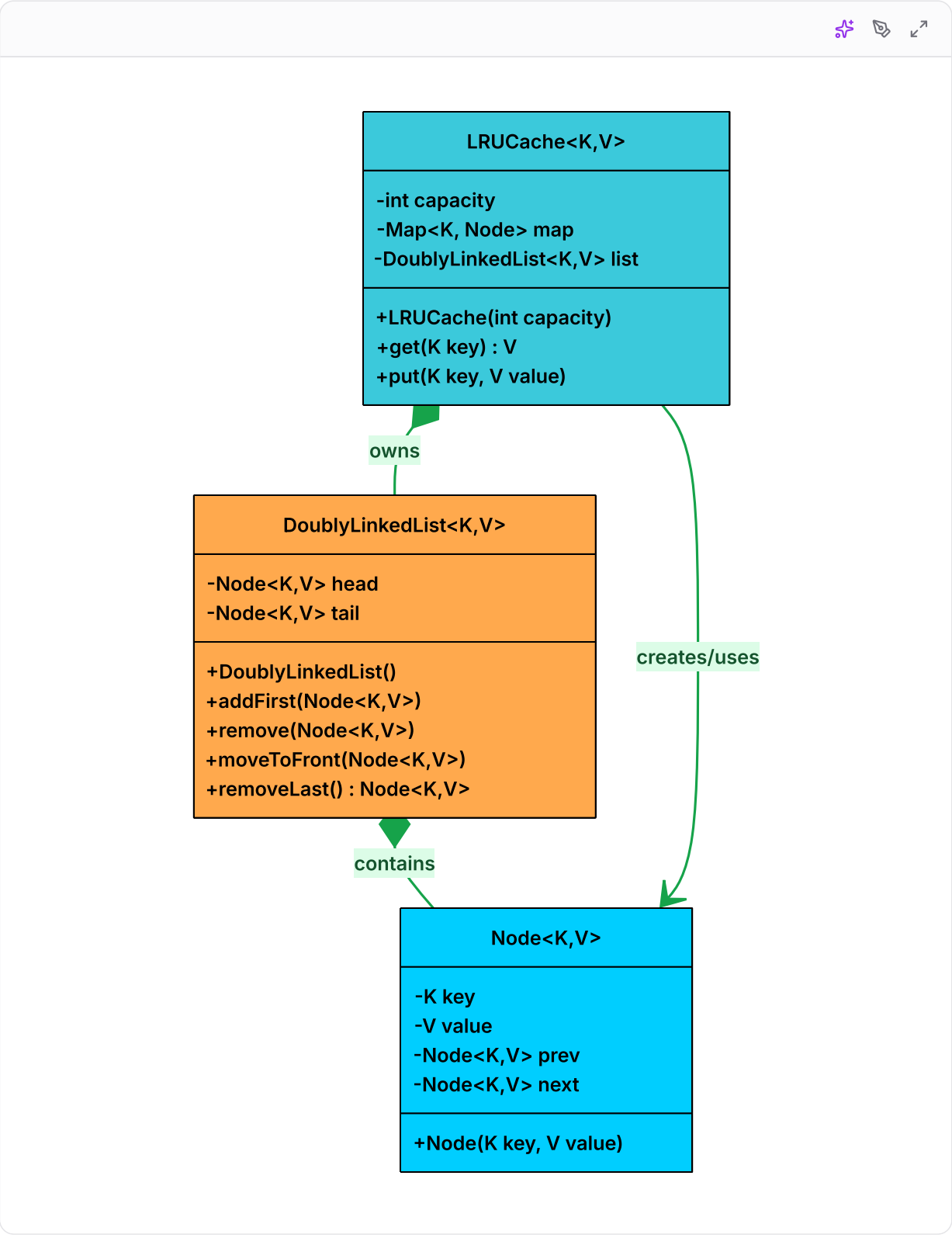
Attribute	Type	Description
`capacity`	int	The maximum number of entries allowed in the cache
`map`	Map<K, Node<K, V>>	Maps keys to their corresponding list nodes for O(1) lookup
`list`	DoublyLinkedList<K, V>	Maintains the recency order of nodes

Method	Description
`LRUCache(capacity)`	Constructor that initializes capacity, creates empty map and list
`get(key)`	Returns value if key exists (and marks as MRU), otherwise returns null
`put(key, value)`	Inserts new entry or updates existing (marks as MRU), evicts LRU if at capacity

Key Design Principles:

1. **Single Responsibility:** The LRUCache coordinates operations but delegates ordering to DoublyLinkedList and lookup to HashMap. Each component does one thing well.
2. **Encapsulation:** The internal data structures are private. Callers only see `get` and `put`. They don't know about nodes, linked lists, or eviction mechanics.
3. **Thread Safety:** Both `get` and `put` should be synchronized to prevent race conditions in multi-threaded environments.

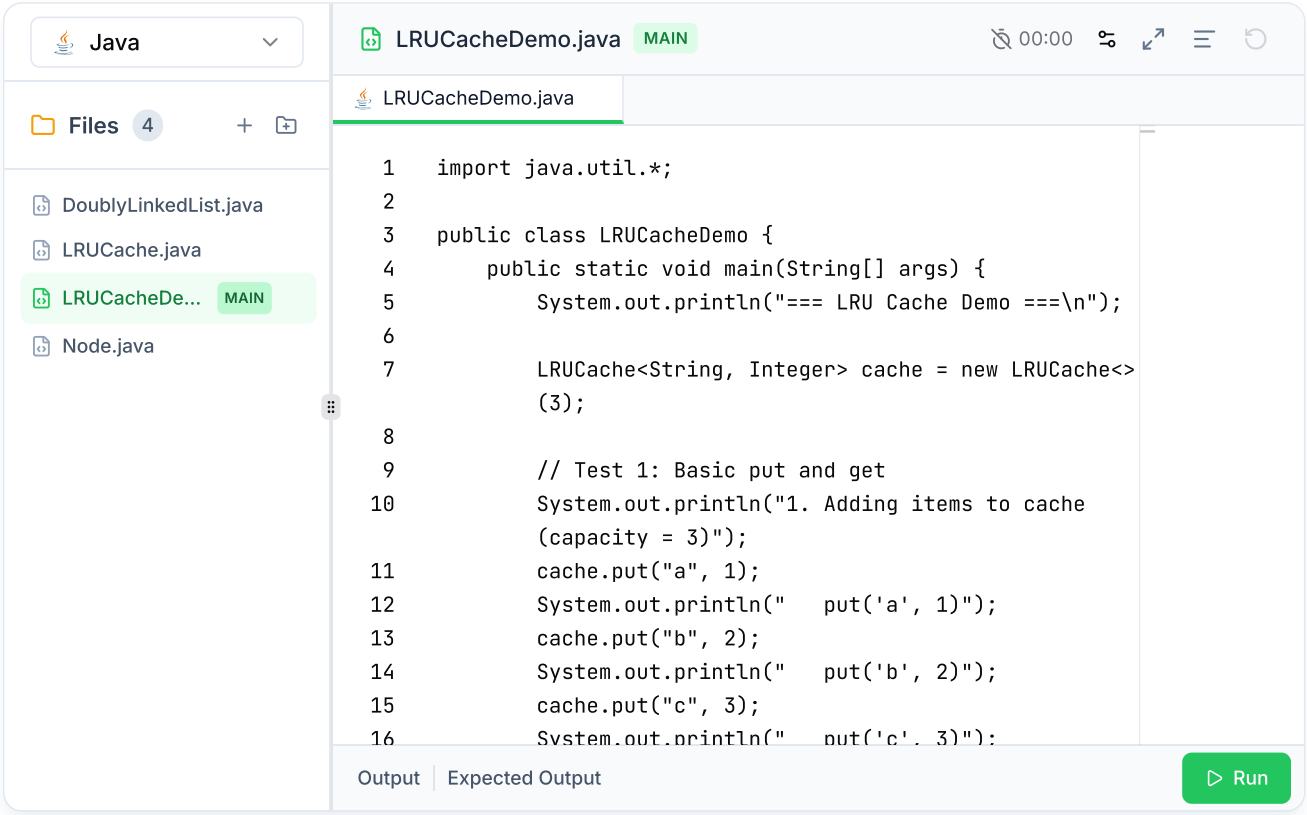
3.2 Full Class Diagram



Try It Yourself (Exercise)

Before looking at the complete implementation, try building the LRU Cache yourself. Below you'll find template stubs for all the classes we designed. Each stub includes method signatures and `// TODO` comments where you need to add the implementation logic.

Your task: Implement all the `// TODO` sections based on the design we discussed. Once complete, run the demo to verify your implementation produces the expected output.



Once you've implemented all the classes and verified the output matches, compare your solution with the complete implementation in the next section.

---

4. Code Implementation

Now let's translate our design into working code. We'll build bottom-up: the Node class first, then the DoublyLinkedList, and finally the LRUCache that ties everything together.

Java

4.1 Node Class

The Node is the fundamental building block. It stores the key-value pair and maintains links to adjacent nodes in the list.

```java
class Node<K, V> {
    K key;
    V value;
    Node<K, V> prev;
    Node<K, V> next;

    Node(K key, V value) {
        this.key = key;
        this.value = value;
    }
}
```

A few things to note:

- **Fields are package-private:** We don't use getters/setters here because the Node is an internal implementation detail, not part of the public API. The DoublyLinkedList needs direct access to `prev` and `next` for O(1) pointer manipulation.
- **Key is stored:** This might seem redundant since the HashMap maps key→node. But during eviction, we remove the tail node from the list and need to remove it from the map too. The map's `remove()` requires the key, so the node must remember it.
- **prev and next start as null:** The DoublyLinkedList will set these when adding the node.

4.2 DoublyLinkedList Class

This class manages the ordering of cache entries. The head represents the most recently used position, and the tail represents the least recently used.

Java

```java
class DoublyLinkedList<K, V> {
    private final Node<K, V> head;
    private final Node<K, V> tail;

    public DoublyLinkedList() {
        // Create dummy nodes
        head = new Node<>(null, null);
        tail = new Node<>(null, null);

        // Link them together
        head.next = tail;
        tail.prev = head;
    }

    public void addFirst(Node<K, V> node) {
        // Insert node between head and head.next
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    public void remove(Node<K, V> node) {
        // Bypass this node by linking its neighbors
        node.prev.next = node.next;
        node.next.prev = node.prev;
```

```
27      }
28
29      public void moveToFront(Node<K, V> node) {
30          remove(node);
31          addFirst(node);
```

Let's walk through each method:

Constructor: Creates dummy head and tail nodes with null keys and values. These dummies never contain real data. They exist to eliminate null checks. After construction, the list looks like: `HEAD <-> TAIL`

addFirst(node): Inserts a node right after the head. The order of pointer updates matters:

1. Set the new node's pointers first
2. Then update the existing nodes' pointers

If we updated `head.next` before using it to set `node.next`, we'd lose the reference.

remove(node): The beauty of doubly linked lists. We don't need to traverse to find the node's neighbors. We have direct references via `node.prev` and `node.next`. We simply make them point to each other, bypassing the removed node.

moveToFront(node): This is the key operation for maintaining LRU order. When an item is accessed, we move it to the front. Rather than implementing special-case logic, we just remove and re-add.

removeLast(): Returns the node just before the tail (the LRU item). We check for empty list by seeing if `tail.prev == head` (meaning no real nodes exist).

4.3 LRUCache Class

This is the main class that ties everything together. It coordinates the HashMap and DoublyLinkedList to provide O(1) operations.

Java

```
1   import java.util.HashMap;
2   import java.util.Map;
3
4   class LRUCache<K, V> {
5       private final int capacity;
6       private final Map<K, Node<K, V>> map;
7       private final DoublyLinkedList<K, V> list;
8
9       public LRUCache(int capacity) {
10          this.capacity = capacity;
11          this.map = new HashMap<>();
12          this.list = new DoublyLinkedList<>();
13
14      public synchronized V get(K key) {
```

```
    public synchronized V get(K key) {
        if (!map.containsKey(key)) {
            return null;
        }

        Node<K, V> node = map.get(key);
        list.moveToFront(node);
        return node.value;
    }

    public synchronized void put(K key, V value) {
        if (map.containsKey(key)) {
            // Update existing entry
            Node<K, V> node = map.get(key);
            node.value = value;
            list.moveToFront(node);
        } else {
```

Let's trace through the logic:

Constructor: Initializes the capacity, creates an empty HashMap, and creates an empty DoublyLinkedList. The cache starts with no entries.

get(key):

1. Check if the key exists in the map. If not, return null.
2. Get the node reference from the map.
3. Move the node to the front of the list (marking it as most recently used).
4. Return the value.

All operations are O(1): HashMap lookup, pointer manipulation in the list.

put(key, value):

Case 1: Key already exists

1. Get the existing node from the map
2. Update its value
3. Move it to the front (accessing counts as usage)

Case 2: Key is new

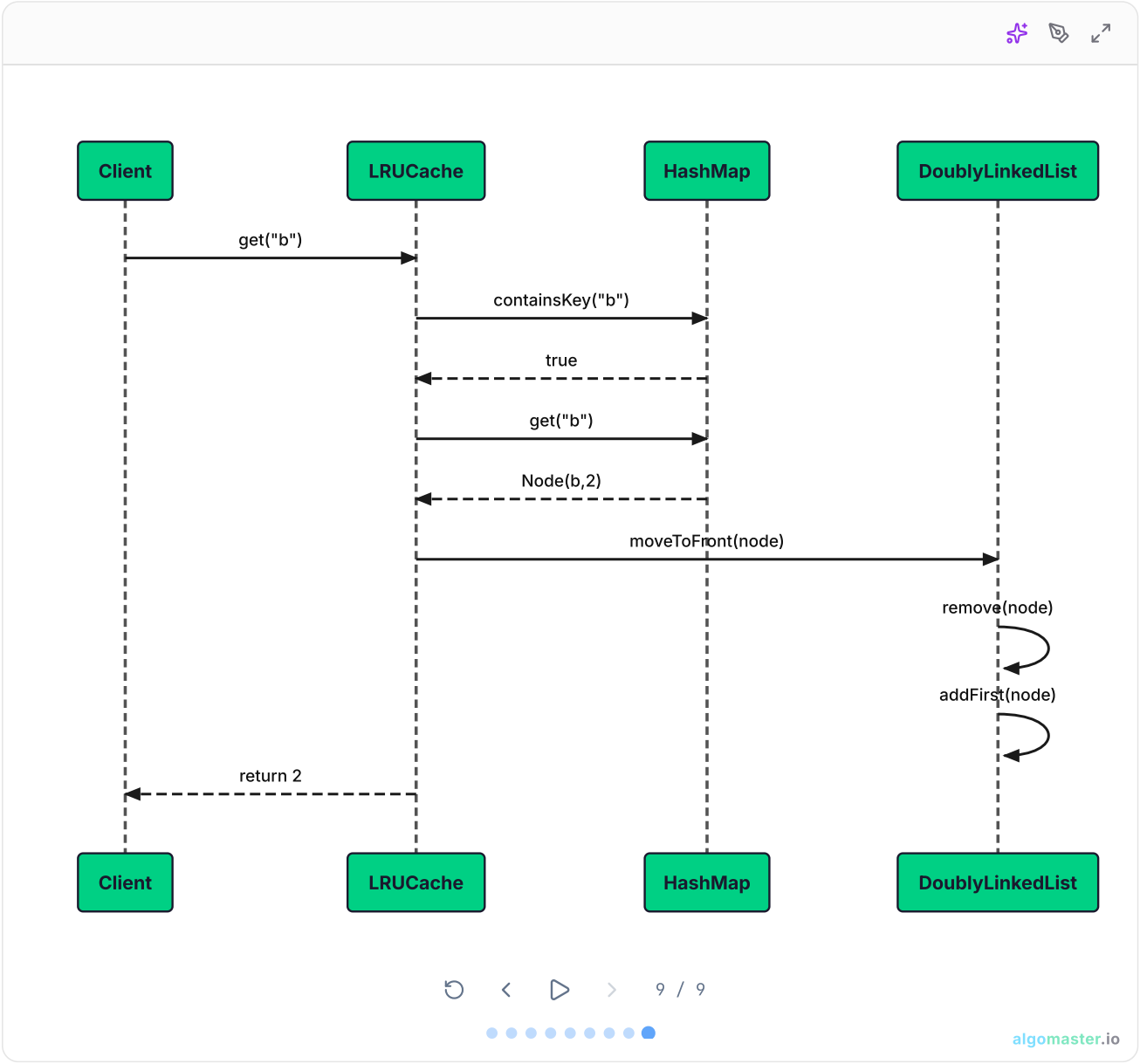
1. Check if we're at capacity. If so, evict the LRU item:
    - Remove the last node from the list
    - Use its key to remove the entry from the map
2. Create a new node with the key and value
3. Add the node to the front of the list
4. Add the key→node mapping to the map

Thread Safety: Both methods are marked `synchronized`. This ensures that only one thread can execute either method at a time on the same cache instance. In a high-throughput scenario, you might use more fine-grained locking, but for interview purposes, `synchronized` demonstrates awareness of concurrency.

Operation Sequence Diagrams

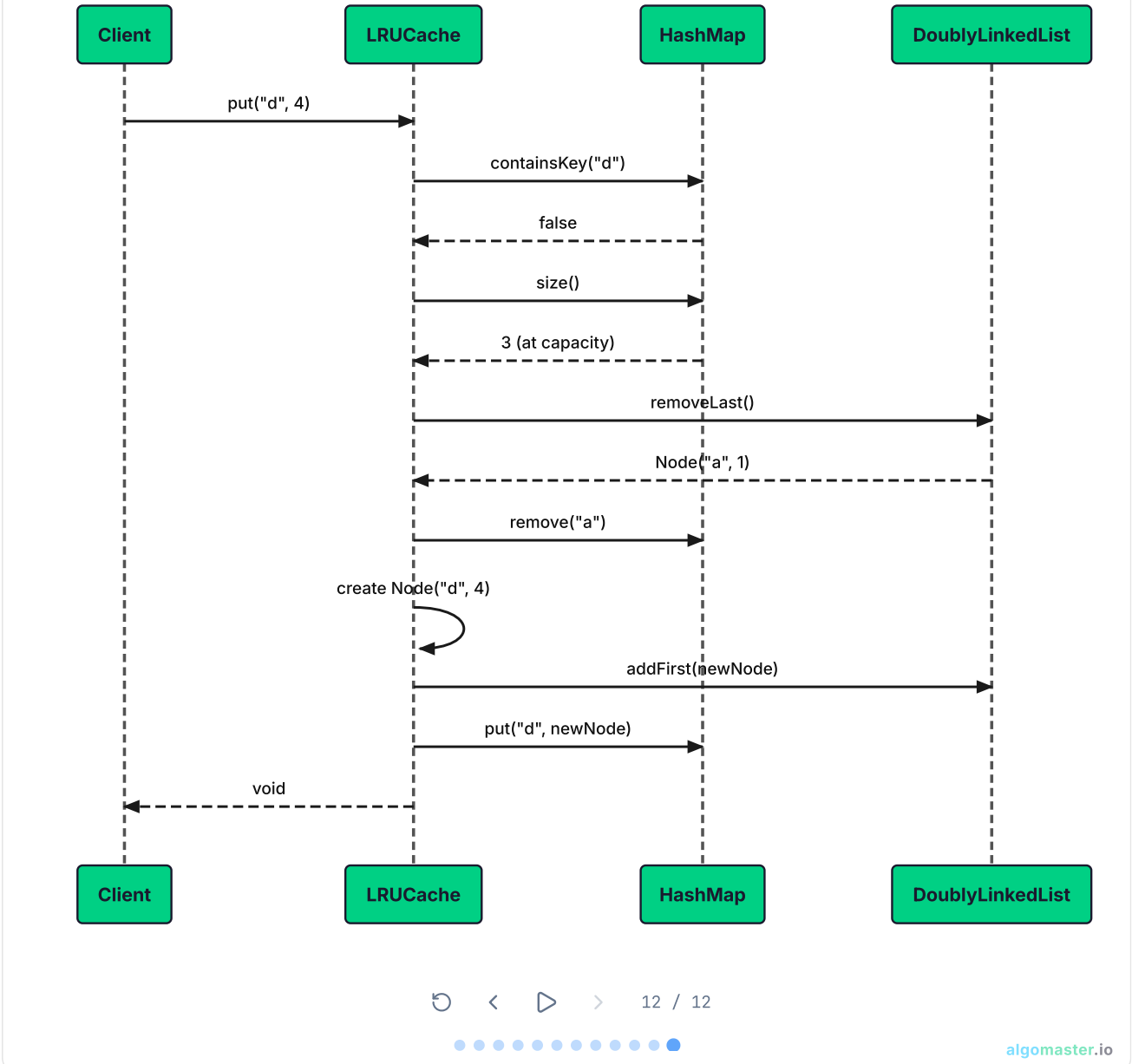
The following diagrams illustrate what happens during get and put operations:

get(key) - Cache Hit

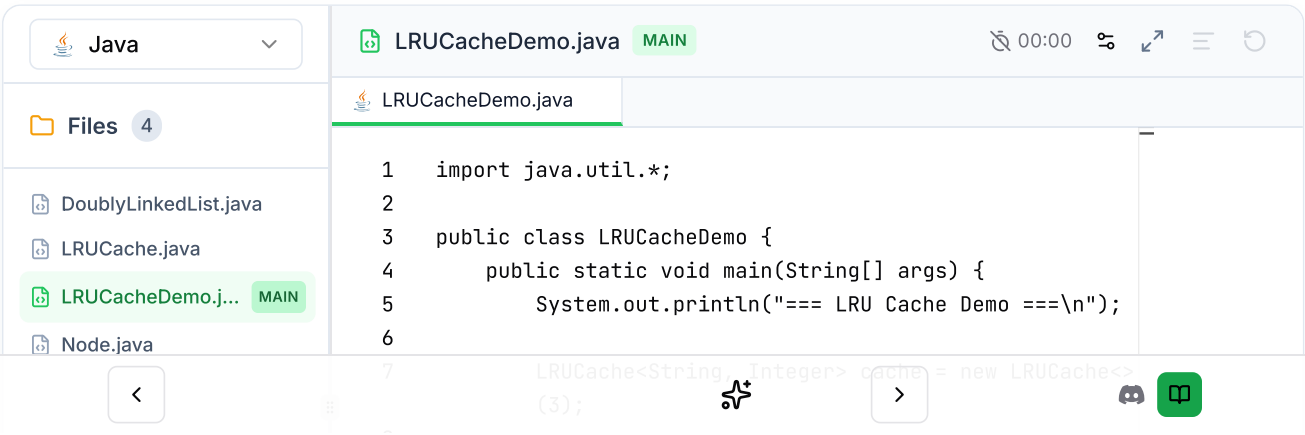


put(key, value) - Eviction





5. Run and Test



```
 8
 9          System.out.println("1. Adding items to cache
             (capacity = 3)");
10          cache.put("a", 1);
11          System.out.println("   put('a', 1)");
12          cache.put("b", 2);
13          System.out.println("   put('b', 2)");
14          cache.put("c", 3);
15          System.out.println("   put('c', 3)");
16          System.out.println("   Cache state: {a=1, b=2
```

Output | Expected Output ▷ Run

6. Concurrency and Thread Safety

Inside a multi-threaded server, many request threads share one `LRUCache` instance and call `get` and `put` concurrently. The cache holds two pieces of state that must stay in sync: the `map` from keys to nodes, and the `list` ordering those nodes from most to least recently used. It is correct only when the map and the list agree.

The implementation wraps both `get` and `put` in a single lock, so only one thread runs either method at a time. The two concerns below show why that lock is needed, and why even `get` cannot run lock-free.

Concern 1: Concurrent Operations Corrupting the Linked List (High Risk)

The list's `prev` and `next` pointers only make sense as a group. `addFirst`, `remove`, and `moveToFront` each rewrite three or four of them in sequence. If two threads run those sequences at once, their half-finished updates interleave and the list points at nodes that have already moved.

Setup

The cache holds `{a, b, c}` ordered `head <-> c <-> b <-> a <-> tail`. Thread 1 calls `get("a")`, running `moveToFront(nodeA)`. Thread 2 calls `put("d", 4)` at capacity, running `removeLast()` to evict `nodeA`, the LRU node just before the tail. Both touch `nodeA` and its neighbors.

Without synchronization

1. Thread 1 (`moveToFront` → `remove(nodeA)`): sets `nodeA.prev.next = nodeA.next`, so `b.next = tail`
2. Thread 2 (`removeLast` → reads `tail.prev`): still reads `nodeA` as the last real node, because Thread 1 has not finished detaching it

3. Thread 2 (`remove(nodeA)`): sets `nodeA.prev.next = nodeA.next` again and `nodeA.next.prev = nodeA.prev`, writing pointers based on a list that Thread 1 is in the middle of changing
4. Thread 1 (`addFirst(nodeA)`): re-inserts `nodeA` at the front, so the node Thread 2 believes it just evicted is now linked back into the list
5. Thread 2 removes `nodeA`'s key from the `map`

Result

The list and the map disagree. `nodeA` is still reachable through the list but its key was deleted from the map, so a later `removeLast()` can return a node whose key is not in the map. `map.remove(lru.key)` then deletes nothing while the list grows past `capacity`, leaking entries and returning stale values.

With synchronization

One lock around the whole `get` and the whole `put` makes each operation atomic. Thread 1 finishes its entire `moveToFront` before Thread 2 acquires the lock, so Thread 2 reads a consistent `tail.prev`, evicts the real LRU node, and removes the matching key. The list and the map move together, one operation at a time.

Java

```java
// the synchronized keyword serializes get and put on the same instance
public synchronized void put(K key, V value) {
    if (map.containsKey(key)) {
        Node<K, V> node = map.get(key);
        node.value = value;
        list.moveToFront(node);
    } else {
        if (map.size() == capacity) {
            Node<K, V> lru = list.removeLast();
            if (lru != null) {
                map.remove(lru.key);
            }
        }
        Node<K, V> newNode = new Node<>(key, value);
        list.addFirst(newNode);
        map.put(key, newNode);
    }
}
```

The lock must cover the map and the list together. Locking only one leaves the same window open, since the corruption comes from the two structures being updated out of step.

⚠ Why a read lock is not enough

It is natural to treat `get` as a read and let many `get` calls run together under a shared read lock. That does not work, because `get` is not a read. Every successful `get` calls

`moveToFront`, which rewrites the list pointers, so two concurrent `get` calls are two writers to the same list and corrupt recency order the same way. Both `get` and `put` must take the same exclusive lock; there is no read-only path to optimize.

Concern 2: A Single Global Lock Becomes the Bottleneck (Medium Risk)

The single lock that fixes Concern 1 is correct, but it serializes every operation. Under heavy load, every `get` and `put` across all keys queues behind one lock, turning the cache into a contention point. This is a scaling problem, not a correctness one.

Setup

Sixteen worker threads run reads and writes spread across thousands of unrelated keys. The work on each key is independent, but all of it shares one lock.

Without striping (single lock)

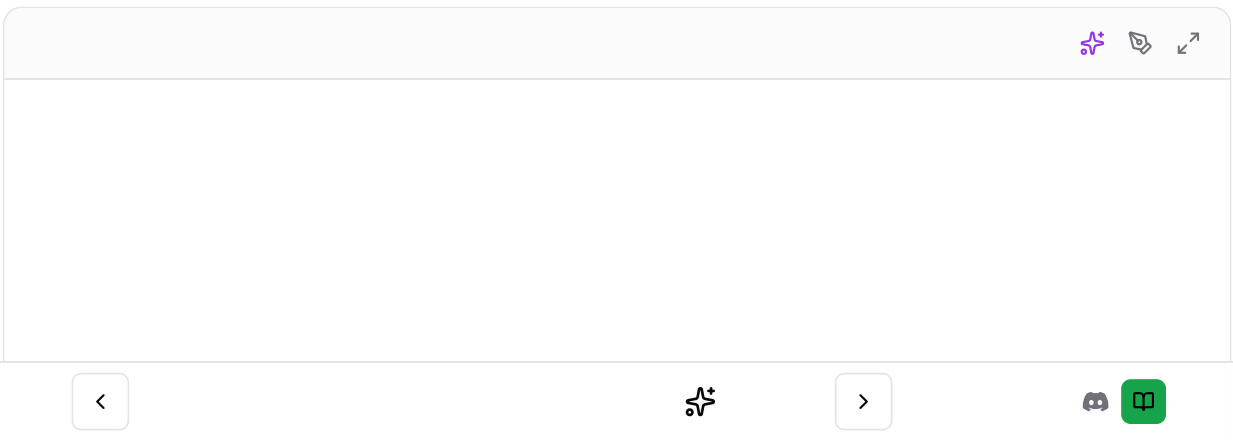
Thread 1 works on `"user_1"` and Thread 2 on `"user_9999"`, keys that never touch the same node, yet Thread 2 still blocks until Thread 1 releases the lock. Throughput is capped at one operation at a time no matter how many cores are available.

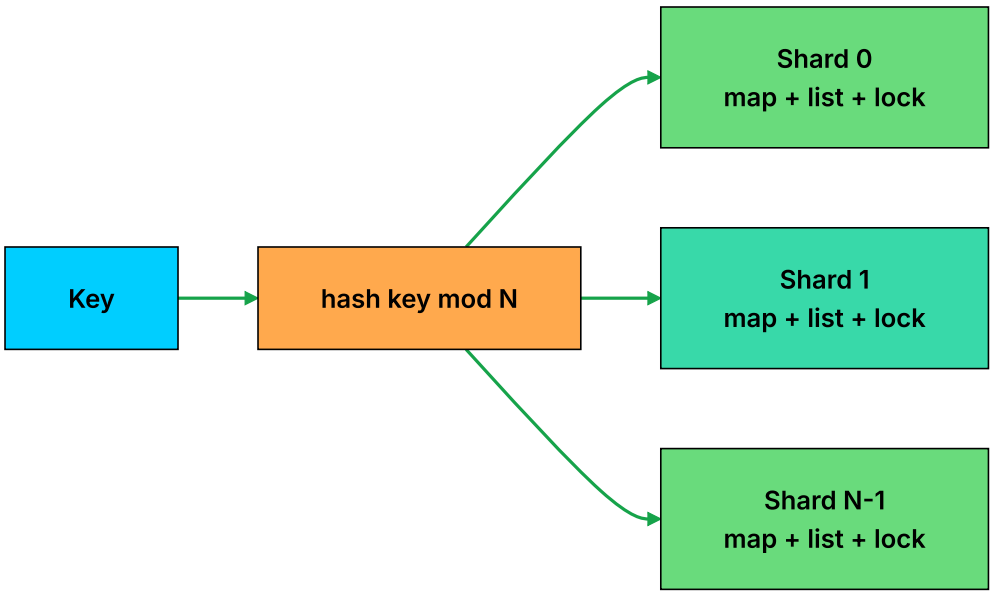
Result

Correct values, but poor scaling. Latency climbs as threads queue on the lock, and adding cores does not help because the critical section is global.

With striped locking

Partition the cache into N independent shards, each with its own map, list, and lock. A key is routed to a shard by its hash, for example `shard = hash(key) % N`. Operations on keys in different shards run in parallel because they take different locks, and each shard runs the same single-lock logic from Concern 1. The trade-off is that eviction is now per-shard rather than global: each shard evicts its own LRU entry, which approximates global LRU closely enough for most workloads. Production caches like Java's `ConcurrentHashMap` and Guava's `Cache` use this approach internally.





Java

```java
// Route each key to one of N shards; each shard is an independent LRUCache
public class StripedLRUCache<K, V> {
    private final LRUCache<K, V>[] shards;

    @SuppressWarnings("unchecked")
    public StripedLRUCache(int shardCount, int perShardCapacity) {
        shards = new LRUCache[shardCount];
        for (int i = 0; i < shardCount; i++) {
            shards[i] = new LRUCache<>(perShardCapacity);
        }
    }

    private LRUCache<K, V> shardFor(K key) {
        int index = (key.hashCode() & 0x7fffffff) % shards.length;
        return shards[index];
    }

    public V get(K key) { return shardFor(key).get(key); }
    public void put(K key, V value) { shardFor(key).put(key, value); }
}
```

These two concerns follow the usual path for shared-state design: first make it correct with a lock that covers the whole operation, including the read path, then trade strict global ordering for throughput by partitioning the lock. Correctness comes from Concern 1; striping is the optimization added once a single lock becomes the bottleneck.

---

7. Extensions

The base cache does one job: keep the most recently used entries and evict the rest. Real caching layers usually need more, such as expiring stale data, capping by memory instead of count, reporting how well the cache performs, and staying consistent with a backing store. Each extension below adds a focused capability without rewriting the core `get` and `put` logic.

7.1 Per-Entry TTL Expiry

Scenario: "Cached values should expire after a fixed time, even if they are still being accessed."

LRU evicts based on recency, but some data goes stale on a clock rather than on usage. A session token or a price quote might be valid for only 60 seconds. The fix is to stamp each node with an expiry time and treat an expired entry as a miss. The cleanest place to enforce this is inside `get`: if the looked-up node is past its expiry, remove it and return null as if it were never there.

Java

```java
// add an expiry field to Node, set when the entry is written
class Node<K, V> {
    K key;
    V value;
    long expiryAtMillis; // System.currentTimeMillis() + ttl
    Node<K, V> prev;
    Node<K, V> next;
}

public synchronized V get(K key) {
    Node<K, V> node = map.get(key);
    if (node == null) {
        return null;
    }
    if (System.currentTimeMillis() >= node.expiryAtMillis) {
        list.remove(node);     // expired: drop it and report a miss
        map.remove(key);
        return null;
    }
    list.moveToFront(node);
    return node.value;
}
```

Expiry on access (lazy expiration) is cheap but leaves dead entries in the cache until someone looks them up. For aggressive memory reclamation, a background sweeper thread can periodically scan and drop expired nodes, at the cost of an extra thread contending for the same lock.

7.2 Weight-Based (Size-Aware) Eviction

Scenario: "Cap the cache by total memory, not by a fixed number of entries."

Counting entries assumes every value is the same size. When the cache holds objects of wildly different sizes (a 2 KB thumbnail next to a 5 MB image), a count limit either wastes memory or

under-uses it. The fix is to give each node a weight and track the running total. Eviction then keeps calling `removeLast` until the total weight fits under the limit, which can remove several small entries or a single large one.

Java

```
private long maxWeight;
private long currentWeight;

public synchronized void put(K key, V value, long weight) {
    if (map.containsKey(key)) {
        Node<K, V> node = map.get(key);
        currentWeight += weight - node.weight;
        node.value = value;
        node.weight = weight;
        list.moveToFront(node);
    } else {
        Node<K, V> newNode = new Node<>(key, value, weight);
        list.addFirst(newNode);
        map.put(key, newNode);
        currentWeight += weight;
    }
    // evict until we fit under the weight budget
    while (currentWeight > maxWeight) {
        Node<K, V> lru = list.removeLast();
        if (lru == null) break;
        map.remove(lru.key);
        currentWeight -= lru.weight;
    }
}
```

The one edge case to handle is a single entry whose weight exceeds `maxWeight` on its own. The loop above would evict everything else and still be over budget. A production cache either rejects such entries or caps weight at the limit, so the cache never tries to store something it can never fit.

7.3 Hit and Miss Metrics

Scenario: "Report how often the cache actually helps."

A cache is only worth its memory if it serves enough requests from memory to justify itself. The hit ratio (hits divided by total lookups) is the single most useful number for tuning capacity. The fix is to increment two counters inside `get`: a hit when the key is present and live, a miss when it is absent or expired. Exposing a small stats snapshot lets a dashboard or log line track the ratio over time.

Java

```
private long hits;
private long misses;

public synchronized V get(K key) {
    Node<K, V> node = map.get(key);
```

```
        if (node == null) {
            misses++;
            return null;
        }
        hits++;
        list.moveToFront(node);
        return node.value;
    }

    public synchronized double hitRatio() {
        long total = hits + misses;
        return total == 0 ? 0.0 : (double) hits / total;
    }
```

Because the counters are updated inside the same locked section as the lookup, they stay accurate under concurrency without a separate lock. A low hit ratio is a signal to either raise the capacity, reconsider what is being cached, or accept that the access pattern does not favor caching at all.

---

8. Quiz



Design LRU Cache Quiz

20 quizzes

Start ›

How helpful was this content?

1 2 3 4 5 6 7 8 9 10

Please sign in to rate this article

Comments (5)

Add a comment...

Sign in to comment

H1 H2 B I U 0/2000 Comment

Search comments...

Sort by Top



Ram Kiran 9 months ago

Too good compared to sites. Thank you Ashish for taking the intiative I recommended my other friends too. Really gr8.

15

R

Raju Sangwan 2 months ago

but bro this initiative is not free, we all have paid for this.

Show 1 reply



Raunak Baliyan 6 days ago

```
func backgroundWorker[K comparable, V any](c *StripedLRUCache[K, V], interval time.Duration) {
  ticker := time.NewTicker(interval)
  defer ticker.Stop()

  for range ticker.C {
    for _, shard := range c.shards {
      shard.mu.Lock()
      now := time.Now()
      for key, node := range shard.mp {
        if !node.expireAt.IsZero() && !now.Before(node.expireAt) {
          shard.list.Remove(node)
          delete(shard.mp, key)
        }
      }
      shard.mu.Unlock()
    }
  }
}
go backgroundWorker(lru, time.Second)
```

1

A

Arya ji 2 months ago

Just Wow ! ❤️ 🎊